Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

TALLER 4: DOCKER

Propósito:

En este taller pondrás en práctica los conceptos fundamentales relacionados con **contenedores y virtualización a nivel de sistema operativo**, utilizando la plataforma Docker como herramienta principal.

A lo largo de la actividad explorarás cómo Docker permite **empaquetar aplicaciones junto con sus dependencias en imágenes reproducibles**, y cómo dichas imágenes pueden ejecutarse como contenedores aislados que comparten el kernel del sistema operativo anfitrión.

Durante el desarrollo del taller trabajarás con comandos básicos de Docker para **descargar imágenes, ejecutar contenedores, inspeccionar su estado y gestionar su ciclo de vida**. Asimismo, analizarás cómo los contenedores representan una abstracción ligera frente a las máquinas virtuales tradicionales y cómo facilitan la portabilidad y el despliegue de aplicaciones.

El objetivo principal es que comprendas que Docker no es únicamente una herramienta de desarrollo moderna, sino también una **tecnología basada en principios fundamentales de los sistemas operativos**, como el aislamiento de procesos, el control de recursos y la compartición eficiente del kernel.

Contexto:

Imagina que debes desplegar una aplicación en distintos entornos: tu computador personal, un servidor institucional y eventualmente una infraestructura en la nube. En cada uno de estos entornos pueden existir diferencias en el sistema operativo, las librerías instaladas o las versiones del software requerido.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Tradicionalmente, este problema se resolvía mediante la creación de máquinas virtuales completas, lo que implica **duplicar sistemas operativos completos y consumir una cantidad significativa de recursos**.

Docker introduce una alternativa basada en **contenedores**, donde las aplicaciones se ejecutan de manera aislada, pero comparten el mismo kernel del sistema operativo anfitrión. Este enfoque permite ejecutar múltiples aplicaciones de manera eficiente, manteniendo entornos reproducibles y fácilmente desplegables.

Durante este taller explorarás cómo las **imágenes de Docker funcionan como plantillas inmutables**, cómo los **contenedores representan instancias en ejecución de dichas imágenes**, y cómo los comandos de Docker permiten gestionar estos recursos en el sistema.

A través de experimentación directa observarás cómo se crean, ejecutan, detienen y eliminan contenedores, comprendiendo así el ciclo de vida completo de una aplicación desplegada mediante contenedores.

Objetivos de aprendizaje:

Este taller tiene como objetivo evaluar la comprensión de los estudiantes sobre los conceptos de concurrencia.

1. Comprender el modelo de **contenedores como mecanismo de virtualización a nivel del sistema operativo**.
2. Identificar los conceptos fundamentales del ecosistema Docker:
    a. imagen
    b. contenedor
    c. registro de imágenes
    d. Dockerfile
3. Ejecutar y gestionar contenedores utilizando comandos básicos de Docker.

4. Analizar la relación entre **imágenes y contenedores**, comprendiendo cómo una imagen funciona como plantilla para crear instancias en ejecución.
5. Construir imágenes simples mediante el uso de **Dockerfile** y comprender su papel en la automatización del despliegue de aplicaciones.

1. Pasos preliminares

Durante el desarrollo de varias de las actividades que se llevarán a cabo en este documento se requerirá del uso de la herramienta Docker. Para llevar las prácticas con la herramienta Docker se sugiere [fuertemente] que se haga el despliegue de Docker en una máquina virtual.

A continuación, se describen algunas de las formas que se pueden utilizar para instalar Docker.

1.1. Vía script

En página web oficial de Docker[1] se indica que para instalar Docker se puede hacer ejecutando los siguientes comandos:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh ./get-docker.sh
```

1.2. Aprovisionando máquina virtual via Vagrant

Para hacer el taller en un ambiente controlado por el momento lo haremos con una máquina virtual, mientras dominamos completamente la herramienta. Vagrant ofrece un comando que permite la instalación de una máquina virtual con Docker listo para

[1] https://docs.docker.com/engine/install/ubuntu/#install-using-the-convenience-script

ser ejecutado. Visitar [este enlace](#)[2]. El archivo que permite hacerlo sería similar al siguiente:

`Vagrantfile` que aprovisiona una máquina virtual con Docker:

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/xenial64"
  config.vm.provider :virtualbox do |vb|
       vb.customize [ 'modifyvm', :id, '--name', 'dockerlocal']
  end
  config.vm.provision "docker"
end
```

1.3. Instalando Docker manualmente ambiente unix[3]

1.
```
sudo apt-get update
```
2.
```
sudo apt-get install \
    curl \
    ca-certificates \
    lsb-release \
    gnupg
```
3.
```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /usr/share/keyrings/docker-archive-keyring.gpg
```
4.
```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
> /dev/null
```

[2] Sección `Pulling Images`

[3] https://docs.docker.com/install/linux/docker-ce/ubuntu/

5.
```
sudo apt-get update
```
6.
```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```
7. Este último paso para adicionar al **$USER** grupo **docker**
```
sudo usermod -aG docker $USER
```

Teniendo como base el enlace anterior cree un Vagrantfile que le servirá de base para próximos despliegues de Docker en este documento.

Tenga en cuenta lo siguiente para su Vagrantfile:

1. Docker solo funciona sobre sistemas operativos Linux de 64 bits ¿Qué debe hacer para crear una máquina virtual con un sistema Linux de 64 bits?
2. Cuando aprovisione la máquina virtual asegúrese de que la máquina virtual tenga un *hostname* similar al nombre que tiene la máquina virtual en el *dashboard* de VirtualBox. **Hint**: Inicialize la variable `config.vm.hostname`[4] (revisar los links es clave para este punto)

1.4. Instalando Docker manualmente ambiente windows[5]

Para Windows se recomienda:

- Docker toolbox para usuarios de Windows 7,8 o 10 Home Edition - No soporta contenedores de Windows

Docker Desktop para usuarios de Windows 10 y Windows Server 2016[6]

Ahora hagámoslo nosotros mismos.

[4] En esta página encontrará detalles de la inicialización de la variable

[5] https://docs.docker.com/install/linux/docker-ce/ubuntu/

[6] Soporta contenedores de Windows.

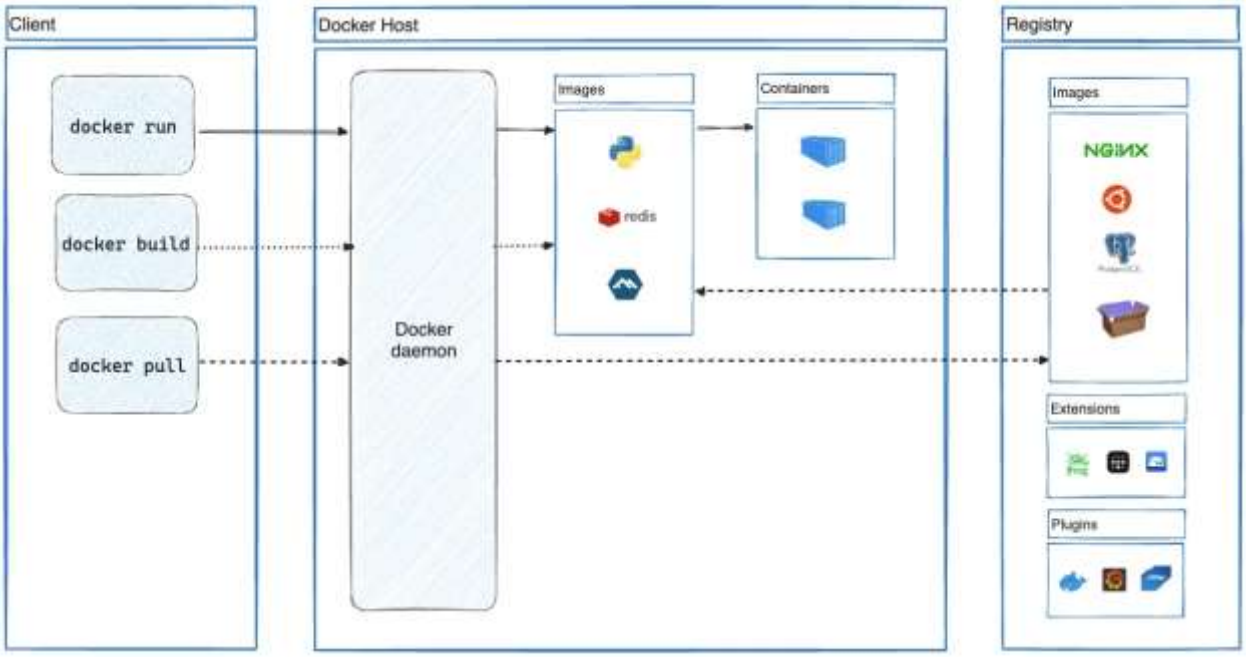
Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

2. Conceptos generales de Docker

Apoyándonos en la presentación realizada en clase y la lista de vídeos de reproducción sobre Docker que nos comparte el profesor John Sanabria podremos resolver los siguientes puntos. Vale la pena mencionar que Docker tiene, además de los conceptos ya mencionados en la presentación, una serie de objetos que permiten la creación de entornos de computación profesional que apoyan el desarrollo de aplicaciones distribuidas. Los objetos por considerar son:

- Image → AMIs
- Container → VM
- Network → VPC (Virtual Private Connectivity)
- Volume → ~~S3~~ EBS

Miremos un ejemplo de la arquitectura de Docker que complementa la vista en la presentación durante la sesión V del curso[7]



[7] Tomada de https://docs.docker.com/get-started/overview/ (2024/03/19)

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

A continuación, se mostrará una especie de *cheat sheet* relacionada con los objetos imágenes de contenedor y contenedor[8].

- Cómo se gestionan las imágenes en Docker:
  - Listar → `docker images`, `docker image ls`
  - Traer imágenes al sistema → `docker pull <image-name>`, `docker run <image-name>`
  - Borramos → `docker rmi <image-name>`
  - Crear imágenes
    - A partir de contenedores → `docker commit <container-id> <new-image-name>`
    - A partir de Dockerfile → `docker build -t <image-name> .`

```
FROM ubuntu

RUN apt update

RUN apt -y install htop
```

    -
- Cómo se gestionan los contenedores
  - Para crear contenedores → `docker container run -it <image-name>`
  - Para listar contenedores → `docker ps [-a]`, `docker container ls [-a]`
  - Para detiener contenedores → `docker container stop <container-id>`
  - Para reiniciar un contenedor → `docker container start <container-id>`,
  - Para conectar a un contenedor que está en ejecución `docker container attach <container-id>`
  - Para eliminar un contenedor tenemos dos formas:
    - Comando explícito → `docker container rm <container-id>`

---

[8] Las imágenes se pueden considerar como aquellos objetos en Docker desde los cuales se pueden crear contenedores. De otro lado, los contenedores se pueden entender como una instancia de una imagen. De forma análoga como un proceso es la instancia de un programa, un contenedor es una instancia de una imagen. Así como un programa es un archivo almacenado en disco duro, una imagen está formada por una serie de archivos almacenados en disco duro y que constituyen las capas de dicha imagen de contenedor.

        - Comando implícito (a la hora de lanzar un contenedor) → `docker container run -it --rm <image-name>`
- Volúmenes en Docker[9] → dispositivos de almacenamiento independientes de los contenedores
    - Es donde se almacenan la información en contenedores. Cuando los contenedores se eliminan, si no tienen un volumen la información no se mantiene y es eliminada con el contenedor
    - Vamos a crear volúmenes
        - A la hora de crear una imagen
        - A la hora de ejecutar una imagen
        - Usando el API de Docker
    - Compartir directorios entre el anfitrión y el contenedor

3. El comando docker y sus sub-comandos

Requerimientos

Para llevar a cabo la ejecución de los próximos comandos se asume que usted tiene acceso a una máquina con Docker instalado como se indicó en los pasos preliminares.

¿De qué se trata esta sección?

En esta sección se hará un recorrido por los comandos más comúnmente usados en Docker.

3.1. hello-world

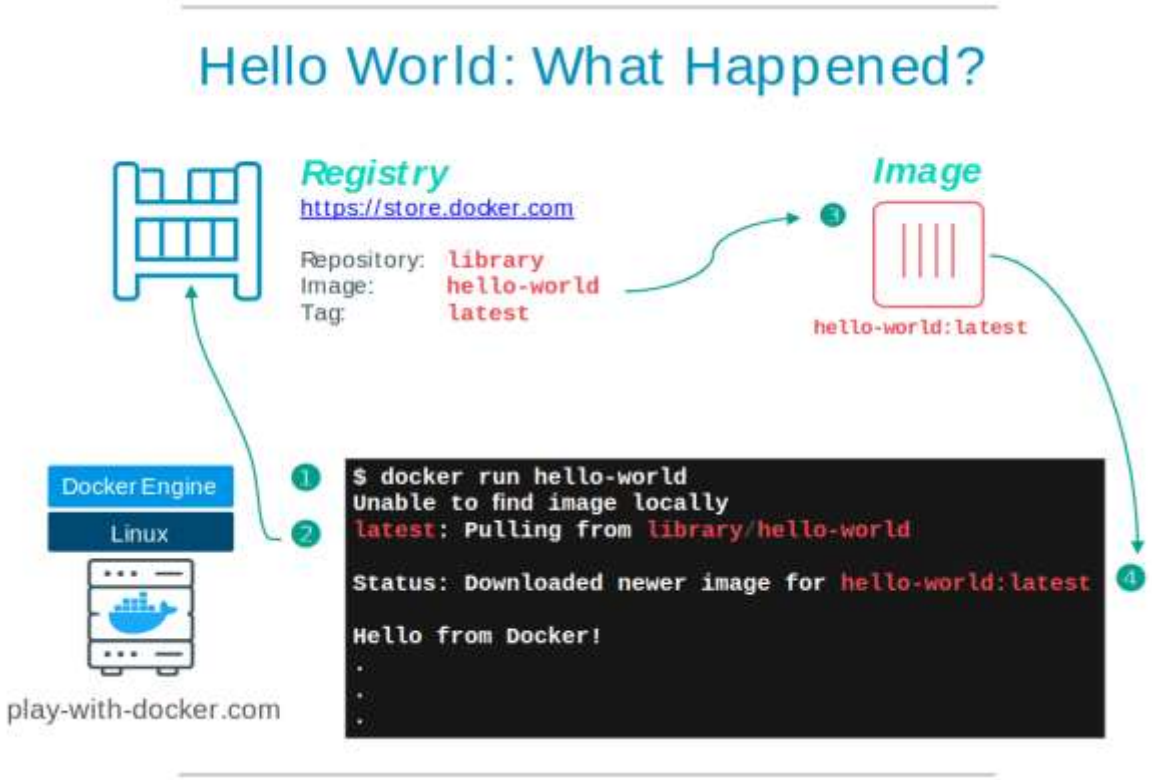
Casi todos los programadores están acostumbrados a su primer programa que imprime el mensaje 'hello world', sigamos entonces está dinámica. En Docker (que **no** es un lenguaje de programación) también tiene su propia versión de 'hello world'. Para ello ejecute el siguiente comando:

[9] El estudio de volúmenes en Docker se toma de este enlace https://training.play-with-docker.com/docker-volumes/

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

```
docker container run hello-world
```

Explicación visual de lo sucedido



¿Cómo lo explicarían con sus propias palabras?

3.2. Docker image pull

`docker`[10] `pull` es un comando que permite la descarga lo que se requiera desde un *registry (repositorio que contiene muchas imágenes preconfiguradas)*

[10] docker es el CLI

Algunas imágenes populares de Docker son:

- Ubuntu - https://hub.docker.com/_/ubuntu/
- Busybox - https://hub.docker.com/_/busybox/

Para descargar imágenes se debe ejecutar:

```
docker image pull ubuntu
```

```
docker image pull busybox
```

Estas imágenes quedan almacenadas localmente en el computador desde donde se emitió el comando `pull`, por tanto debe tener espacio de disco duro suficiente para hacerlo.

El sitio para buscar repositorios es https://hub.docker.com[11].

Ponlo en práctica

Visite https://hub.docker.com, busque alguna aplicación *Open Source* que le interese (base de datos, servidor web, etc.) y valide si existe una imagen de Docker para esta aplicación. De ser así, descárguela.

[11] http://hub.docker.com es el repositorio en Internet de imágenes de Docker. La plataforma de Docker está constituida por el CLI + Docker hub.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.3. Docker images

Recordemos que una imagen en Docker es un archivo inmutable que es fundamentalmente una “fotografía” de un contenedor. Las imágenes de Docker son la base para la ejecución de contenedores. Es similar a la analogía que una aplicación es un archivo en un dispositivo de almacenamiento pero que una vez este se ejecuta se convierte en un proceso.

El subcomando `image ls` es usado para listar las imágenes de Docker que están disponibles de forma local.

```
docker image ls
```

Ponlo en práctica

Cree un alias en Bash (comando corto que en realidad ejecuta otro comando más largo) que le permita contar cuantas imágenes de Docker tiene en su computador. Un ejemplo simple para compenderlo es el siguiente:

Si creo un alias ll="ls -l" cada vez que ingres en consola ll, en realidad ejecutara la instrucción ls -l.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.4. Docker container run

`docker run` es el comando que permite llevar a cabo la ejecución de contenedores. Se asume que usted ha descargado la imagen ubuntu para Docker. Para llevar a cabo la ejecución de un contenedor de ubuntu ejecute:

```
docker container run busybox[12]
```

Cuando usted corrió este comando, esto fue lo que sucedió

1. Docker buscó una imagen llamada busybox
2. Cargo esa imagen y creó un contenedor
3. Finalmente, se ejecutó el contenedor

Sin embargo, no tenía nada que ejecutar.

Ahora ejecute lo siguiente

```
docker container run busybox echo “hello world”
```

En esta oportunidad sí se llevó a cabo la ejecución del comando `‘echo’`. Lo que sucedió aquí fue algo como la ejecución de una máquina virtual y dentro de ella se ejecutó el comando (`‘echo’`) y luego esta máquina dejó de existir.

```
docker container run -it busybox sh
```

En esta oportunidad se lanza la ejecución de un contenedor basado en busybox y ejecuta el shell `‘sh’`.

```
docker container run alpine ls -l
```

[12] https://hub.docker.com/_/busybox

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

DIY Ponlo en práctica

Visite el contenedor, observe por ejemplo:

- ¿Qué archivos + directorios tiene este contenedor?
- ¿Qué usuario es usted en ese momento?
- ¿Qué sucede si ejecuta el comando '`rm -rf /bin/*`'? ¿Qué comandos ya no puede ejecutar?
- Abra otra terminal y vuelva a correr el comando '`docker run -it busybox sh`'. El directorio '`/bin`' tiene archivos?
- ¿Qué significa el siguiente comando?

```
docker container run -it ubuntu /bin/bash
```

- Ejecute el comando anterior y:
  - Cree un usuario (https://www.digitalocean.com/community/tutorials/how-to-add-and-delete-users-on-ubuntu-20-04)
  - Instale una aplicación o una base de datos y valide que opera normalmente

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.5. Docker ps

El comando `docker ps` permite ver que contenedores están en ejecución. Sin cerrar la ejecución del contenedor del item anterior, abra otra terminal y ejecute el comando

```
docker ps
```

¿Qué salida muestra dicho comando? Este comando muestra el identificador de todos los contenedores que están corriendo en este momento.

¿Qué salida muestra este comando?

```
docker ps -a
```

Este comando muestra la lista de todos los contenedores que están corriendo o han corrido en el sistema.

Nota: Se sugiere usar el comando `docker container ls` y `docker container ls -a`.

Ponlo en práctica

- Defina un alias en Bash en el cual se lista los **identificadores** de contenedores que están en ejecución. Revisar la página de docker ps.
- Utilice los mismos argumentos que usa para el comando 'ps' pero con el comando 'docker container ls <argumentos_aqui>'.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.6. Docker container rm

Este comando permite borrar un contenedor. Para llevar a cabo esta tarea se hace necesario que ese contenedor ya no esté en ejecución. Para ejecutar este comando se le indica al comando el identificador del contenedor que se desea borrar.

Ponlo en práctica

Hace un momento usted fue capaz de listar los identificadores de aquellos contenedores que estaban corriendo. Construya, en una sola línea de ejecución (alias), un comando que borre todos aquellos contenedores que han terminado su ejecución.

Hint:

- Para llevar a cabo este comando usted deberá invocar el comando `docker container rm <argumentos>`. A continuación, se explicará de donde salen esos `<argumentos>`
- La lista de `<argumentos>` es el resultado de ejecutar el comando docker ps teniendo en cuenta que
  - El comando debe arrojar solo los indicadores (ids) de los contenedores
  - Los indicadores que se deben mostrar es de solo aquellos contenedores que terminaron o **salieron**[13] de su ejecución[14].
- Una vez identifique el comando que muestra los ids de aquellos contenedores que terminaron su ejecución encerrar dicha instrucción en la construcción del alias Bash `$(comando)`
- `$(comando)` se debe pasar como argumento al comando `docker container rm`

[13] exited.

[14] Revisar el comando docker ps el argumento `-f`.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.7. Docker image rm

Este comando permite llevar a cabo la eliminación de la imagen en Docker. Recuerde que a partir de una imagen es que se genera la ejecución de contenedores. Una vez borrada una imagen esta queda inaccesible de forma local, pero puede ser descargada desde Internet si esta se encuentra disponible (en el Docker hub). El argumento que debe recibir este comando es el id de la imagen que se desea borrar.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.8. Docker container stop

`docker stop` es un comando que permite detener la ejecución de un contenedor.

```
docker container stop <container-id>
```

- `Truco`: otra forma de salirse o detener de la ejecución de un contenedor es, estando dentro del contenedor, ejecutar el comando `exit`.
- `Truco`: es posible salirse del contenedor sin detener su ejecución a través del comando `ctrl-p ctrl-q`

Ponlo en práctica

1. Abra una terminal y ejecute un shell en un contenedor
2. Abra otra terminal y ejecute el comando `docker stop` sobre el contenedor que acabó de ejecutar

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.9. Docker container start

El comando `docker start` permite retomar la ejecución de un contenedor que se ha detenido (`docker container stop`) anteriormente. Para validar que contenedores han sido ejecutados con anterioridad y que en este momento no se encuentran en ejecución, se debe ejecutar el comando

```
docker container ls -a
```

Tome el identificador de alguna de esas imágenes y ejecute el comando

```
docker container start <container-id>
```

En este momento el contenedor ha sido reactivado.

Ponlo en práctica

En el ejercicio anterior usted detuvo la ejecución de un contenedor que ejecutaba el shell Bash. Reanude la ejecución de dicho contenedor.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.10. Docker container attach

Este comando permite conectarse a un contenedor o que está en ejecución o que fue reactivado con el comando docker start.
Validar el identificador de un contenedor que esté en ejecución. Ahora, ejecutar el comando

```
docker container attach <running-container-id>
```

Bonus

Para salir de un contenedor se pueden ejecutar la secuencia de teclas `ctrl-p + ctrl-q`

Ponlo en práctica

1. Lance la ejecución de un contenedor
2. Estando al interior del contenedor ejecute la instalación del comando htop
3. Ejecute el comando htop
4. Desconéctese del contenedor haciendo uso de la secuencia ctrl-p + ctrl-q
5. Detenga la ejecución del contenedor
6. Reinicie la ejecución del contenedor
7. Reconéctese al contenedor y valide si el comando htop continúa en ejecución

Algunos elementos para repasar posterior al taller sobre la terminología

Docker maneja una serie de nuevos conceptos que vale la pena tener claros. En este enlace usted encontrará algunos de dichos términos que vale la pena dejar claros. *De este punto no necesita evidencia, pero le servirá para la próxima sesión*

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.11. Un sitio web estático

En esta sección vamos a ejecutar una imagen de Docker que contiene un sitio web estático. A través de este ejemplo se podrá comprender:

- El uso de imágenes de Docker de terceros
- El mapeo de puertos

Accediendo a una imagen de un tercero

1. Descargue la imagen de Docker localizada en `prakhar1989/static-site`. (recuerde la encuentra en Docker hub)
2. Valide que la imagen fue descargada.
3. Ejecute dicha imagen. Asegúrese que una vez termine la ejecución ese contenedor será removido por el cliente de Docker

```
Nginx is running...
```

El problema con esta ejecución es que no se sabe cómo se puede acceder a la aplicación. Un comando que permite validar que puerto ha sido expuesto por un contenedor es el comando `docker port <container-id>.`
Valide si hay algún puerto expuesto por este contenedor. Comente lo que encontró.
Detenga la ejecución del contenedor.

Ahora, intente con esta nueva ejecución

```
docker container run --rm -d -P --name static-site prakhar1989/static-site
```

¿Qué significan:

- `-d`?
- `-P`?
- `--name static-site`?

Ejecute nuevamente el comando `docker container port <container-id>.` En esta ocasión usted puede cambiar el `<container-id>` por `static-site`.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Ponlo en práctica

- Con la información obtenida, acceda al sitio estático desde su navegador.
- Detenga la ejecución del servidor web.
- Intente ingresar nuevamente al sitio web estático.

3.12. Creando su propio servidor web

Usted va a crear un contenedor que corra el servidor nginx y despliegue una página HTML personalizada.

Ponlo en práctica

- Cree un nuevo directorio llamado `myweb`
- Ingrese a ese directorio y cree un archivo llamado `Dockerfile` con la siguiente información

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html

EXPOSE 80 443

CMD ["nginx", "-g", "daemon off;"]
```

- Cree también un archivo llamado `index.html` que contiene la siguiente información. Cambie el XXX por sus nombres

```
<html>
  <body>
       <h1> Saludos, XXX </h1>
  </body>
</html>
```

- En el directorio donde creó estos archivos ejecute el siguiente comando. Antes de dar ENTER asegúrese de inicializar la variable `${MYDOCKERID}` con el login de su usuario en la plataforma http://hub.docker.com . Esto permitirá subir la imagen a su Docker hub.

```
docker image build -t ${MYDOCKERID}/mynginx:0.0.9 .
```

- Pruebe que su contenedor está bien accediendo a el desde su navegador

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

3.13. Compartiendo archivos entre anfitrión e invitado

- Estando en una sesión de terminal en el equipo anfitrión cree una carpeta llamada `HTML`.
- En esa carpeta cree un archivo llamado `index.html`, `nano HTML/index.html`.
- Ingrese la siguiente información en dicho archivo:

```
<html>
  <body>
       <h1 align=center> Hello world</h1>
  </body>
</html>
```

- Salga de la edición del archivo anterior.
- Ejecute ahora el siguiente comando:

```
docker container run -p 8080:80 -v $(pwd)/HTML:/usr/share/nginx/html nginx
```

- Abra una pestaña en su navegador en la dirección http://localhost:8080, debería ver un pantallazo similar a este:

hola mundo

- Modifique ahora el archivo `HTML/index.html` y en lugar de `'hola mundo'` escriba `'hello world'`. Recargue la página, ahora debería ver algo como esto:

hello world

Ponlo en práctica

- Publique su contenedor en docker hub

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

4. Creación de imágenes en Docker vía Dockerfile[15]

El Dockerfile es un insumo que se le pasa al comando `docker build` para construir imágenes de contenedor.

"The docker build command builds an image from a `Dockerfile` and a *context*"

¿Qué es un contexto?

"The build's context is the set of files at a specified location `PATH` or `URL`"

Ejemplo

Dockerfile

```
FROM ubuntu
RUN apt update
RUN apt install -y htop
```

Comando

```
docker build -t <nombre_imagen> .
```

Notas:

- Es posible pasar varios tags (`-t`) a la hora de crear una imagen.
- Docker tiene un caché que permite la rápida ejecución de la construcción de imágenes de contenedor cada vez que lo pueda hacer.

El formato de las instrucciones en un `Dockerfile` es:

```
INSTRUCTION arguments
```

[15] Tomado de https://docs.docker.com/engine/reference/builder/

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Se tiene por costumbre escribir `INSTRUCTION` en mayúsculas.

FROM

Un `Dockerfile` debe comenzar siempre con la instrucción `FROM`. Esta instrucción indica la nueva imagen a partir de qué imagen pre-existente se va a construir.

RUN

Esta instrucción lleva a cabo la ejecución de un comando. Ese comando se escribe de la misma manera como se escribe un comando desde la línea de comandos.

```
RUN apt update
```

Ponlo en práctica

Escriba un Dockerfile que genere una imagen de contenedor con apache instalado en ella. Que la imagen lleve por nombre 'apacheubuntu'.

CMD

El propósito de la instrucción CMD es proveer una ejecución por defecto en un contenedor.

Ponlo en práctica

Escriba las siguientes líneas en un `Dockerfile`.

```
FROM ubuntu
CMD ["ls", "-l"]
```

Construya una nueva imagen de contenedor. Esta imagen deberá llamarse `ls`.

1. Ejecute los siguientes comandos
    a. `docker run -it ls`
    b. `docker run -it ls bash`
2. Describa en sus propias palabras, cuál es la función de la instrucción `CMD` que observo.

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

Nota: De la instrucción `CMD` solo puede haber una por `Dockerfile`.

EXPOSE

Esta instrucción indica que el contenedor estará escuchando en un puerto determinado en tiempo de ejecución.

Esta instrucción no publica el puerto sino que más bien sirve de documentación a quien usa la imagen para que sepa que puerto debe ser publicado.

Para publicar el puerto es necesario pasar como argumento `-p` o `-P` al comando `docker run`.

ADD

Esta instrucción permite copiar archivos o del sistema de fichero local o de archivos remotos en un URL. Este comando permite cambiar los permisos de los archivos a ser copiados.

```
ADD --chown=<user>:<group> <src> <dest>
```

Ponlo en práctica

1. Cree un archivo tgz
2. Construya un Dockerfile que usa la instrucción ADD usando como fuente el archivo creado en el ítem anterior
3. Construya una imagen a partir del Dockerfile definido en el ítem anterior
4. Ejecuta un contenedor a partir de la imagen creada en el ítem anterior
5. ¿Qué puede observar?

COPY

Esta instrucción es bastante similar a `ADD`. La diferencia más importante es que `<src>` no puede ser un URL.

Código: 750001C
Curso: Sistemas Operativos
Periodo: 2026-1 junio -febrero
Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

ARG

Es una instrucción que permite definir variables que se usarán posteriormente durante el proceso de construcción de la imagen.

```
FROM ubuntu
ARG var1="hola"
ARG var2
RUN echo "Saludo ${var1} ${var2}" > /demo.txt
```

Ponlo en práctica

1. Primera parte
    a. Construya una imagen con un Dockerfile conteniendo las líneas descritas en la caja de texto anterior
    b. Ejecute un contenedor a partir de la imagen creada en el ítem anterior. Revise el contenido del archivo 'demo.txt'
2. Segunda parte
    c. Construya una imagen con un Dockerfile conteniendo las líneas descritas en la caja de texto anterior y con el siguiente comando 'docker build --build-arg var2="mundo"'
    d. Ejecute un contenedor a partir de la imagen creada en el ítem anterior. Revise el contenido del archivo 'demo.txt'

ENV

Es una instrucción que permite definir variables que se usarán posteriormente durante el proceso de ejecución de un contenedor.

```
FROM ubuntu
ENV var1="hola"
ENV var2 mundo
RUN echo "Saludo ${var1} ${var2}"
```

Ponlo en práctica

1. Construya una imagen con un Dockerfile conteniendo las líneas descritas en la caja de texto anterior

Profesor: Manuel Alejandro Pastrana, PhD
email: manuel.pastrana@correounivalle.edu.co
Cel: +573007918725

2. Ejecute un contenedor a partir de la imagen construida en el ítem anterior de la siguiente manera ‘docker run --rm -it <nombre_imagen> env’
3. Ejecute un contenedor a partir de la imagen construida en el ítem anterior (y de la forma como se mostró anteriormente) pero adicione los siguientes argumentos al comando ‘-e “var1=hello” -e “var2=”world”’

ENTRYPOINT

Esta instrucción permite configurar un contenedor para que se corra como una aplicación.

```
FROM debian:stable
RUN apt-get update && apt-get install -y --force-yes apache2
EXPOSE 80 443
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```